Intraday Volume Lab

Technical project report

Bikash Kushwaha

A reproducible workflow for next-minute interval-volume forecasting, with causal data preparation, chronological evaluation, and a readable static research report.

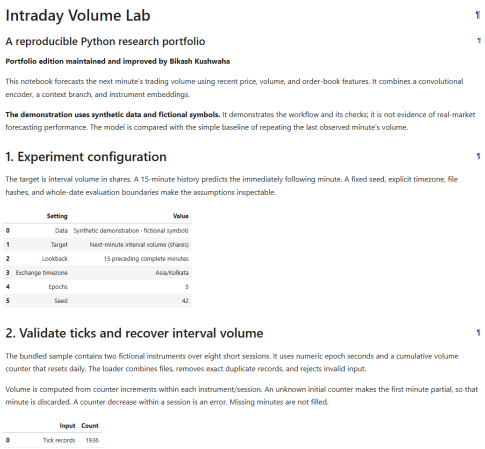
Project purpose

Given the previous 15 complete one-minute bars for one instrument, predict the following minute's traded shares. The implementation combines a convolutional sequence encoder, six context summaries, and an instrument embedding.

Demonstration scope	Recorded result
Synthetic ALPHA and BETA instruments; eight short 60-minute sessions	720 forecasting windows: 450 train, 90 validation, 180 test
Five training epochs; fixed seed 42	Test MAE: CNN 165.19 shares; persistence 94.79 shares

Main finding

Persistence performs better in this run. The demonstration verifies a complete research workflow; it does not establish real-market forecasting performance.



Intraday Volume Lab ¶

A reproducible Python research portfolio ¶

Portfolio edition maintained and improved by Bikash Kushwaha

This notebook forecasts the next minute's trading volume using recent price, volume, and order-book features. It combines a convolutional encoder, a context branch, and instrument embeddings.

The demonstration uses synthetic data and fictional symbols. It demonstrates the workflow and its checks; it is not evidence of real-market forecasting performance. The model is compared with the simple baseline of repeating the last observed minute's volume.

1. Experiment configuration ¶

The target is interval volume in shares. A 15-minute history predicts the immediately following minute. A fixed seed, explicit timezone, file hashes, and whole-date evaluation boundaries make the assumptions inspectable.

	Setting	Value
0	Data	Synthetic demonstration - fictional symbols
1	Target	Next-minute interval volume (shares)
2	Lookback	15 preceding complete minutes
3	Exchange timezone	Asia/Kolkata
4	Epochs	5
5	Seed	42

2. Validate ticks and recover interval volume ¶

The bundled sample contains two fictional instruments over eight short sessions. It uses numeric epoch seconds and a cumulative volume counter that resets daily. The loader combines files, removes exact duplicate records, and rejects invalid input.

Volume is computed from counter increments within each instrument/session. An unknown initial counter makes the first minute partial, so that minute is discarded. A counter decrease within a session is an error. Missing minutes are not filled.

	Input	Count
0	Tick records	1936

Figure 1. The deployed report presents the experiment, validation policy, and recorded results.

02 / PROJECT REPORT

Data preparation and architecture

The notebook explains the experiment. The shared **volume_lab.py** module owns the numerical implementation and command-line interface, so ingestion, training, and evaluation do not depend on duplicated notebook logic.

Stage	Responsibility
Load and validate	Combine JSON-lines files by instrument; remove exact duplicates; validate schema, numerical values, timestamps, and quotes.
Recover interval volume	Convert cumulative session counters into increments; aggregate complete minutes; exclude unallocated partial observations.
Build examples	Create 15-minute consecutive windows, context summaries, instrument IDs, and next-minute targets.
Split and preprocess	Apply shared whole-date boundaries; fit feature and target scalers on training only.
Train, evaluate, publish	Fit the CNN, compare persistence, save provenance and model artifacts, and export a static report.

Input contract

Each record provides timestamp, symbol, ltp, bid_price, ask_price, bid_size, ask_size, and vol_traded_today. Prices must be positive and finite, sizes and cumulative shares non-negative, and ask must be at least bid. Timestamp units are explicit seconds or milliseconds; the default exchange timezone is Asia/Kolkata.

Counter and gap policy

Counters are differenced within each symbol and local calendar date. A zero first counter is a known baseline. A positive first counter leaves its increment unknown, so the partial initial minute is discarded. A decrease inside a session raises an error; a new date allows a normal daily reset.

Missing minutes are not filled or bridged. The first observed minute after a feed gap is also discarded because the counter jump contains volume that cannot be allocated safely. Windows remain within a single session and require complete consecutive minutes.

Implementation references: data contract and shared module.

03 / PROJECT REPORT

Model design and evaluation policy

Input branch	Features and transformation
Sequential: 15 x 5	Close, interval volume, mean spread, bid/ask imbalance, and high-low range. Conv1D 64 then 32 filters, kernel 3, ReLU, layer normalization, and flattening.
Context: 6 values	Mean close, close standard deviation, summed volume, mean spread, mean imbalance, and window range. Dense 32 and layer normalization.
Instrument ID	Training-derived vocabulary and an eight-dimensional embedding, flattened before concatenation.

Regression head and training

Concatenated representations feed dropout (0.2), Dense 128 with layer normalization, another dropout (0.2), Dense 64, and one output. Training uses MSE, Adam at 0.0005, gradient clipping at 1, L2 regularization at 0.0005, checkpointing, and early stopping. The recorded demonstration completes five epochs with seed 42.

The target uses log1p(shares), then StandardScaler fitted on training samples. Results invert the scaling and report errors in share units as well as log-volume units. Negative predicted log values are clipped to zero when converting to non-negative shares.

Chronological partitions

Partition	January 2026 dates	Windows
Training	05-09 (five dates)	450
Validation	12 (one date)	90
Test	13-14 (two dates)	180

All instruments use the same date boundaries. Evaluation instruments must have appeared in training. Random shuffling applies only within the selected training partition. Whole-date separation prevents training on another instrument from a later evaluation date.

The six context features are aggregate summaries, not a CMEM estimator. This architecture is not claimed to reproduce either referenced research model in full.

04 / PROJECT REPORT

Recorded results and interpretation

The included sample contains 1,936 ticks, 960 complete minute bars, and 720 forecasting windows. The following results use the 180 held-out test windows from two dates. Persistence predicts the most recent minute's volume.

Share-unit metric	CNN	Persistence
MAE (shares)	165.19	94.79
RMSE (shares)	220.78	128.84
R-squared	0.619	0.870
SMAPE (%)	16.86	9.16

Persistence has lower MAE and RMSE and higher R-squared. The CNN does not outperform this baseline on the recorded synthetic run. A small sample and five epochs limit what can be concluded about model selection or generalization.

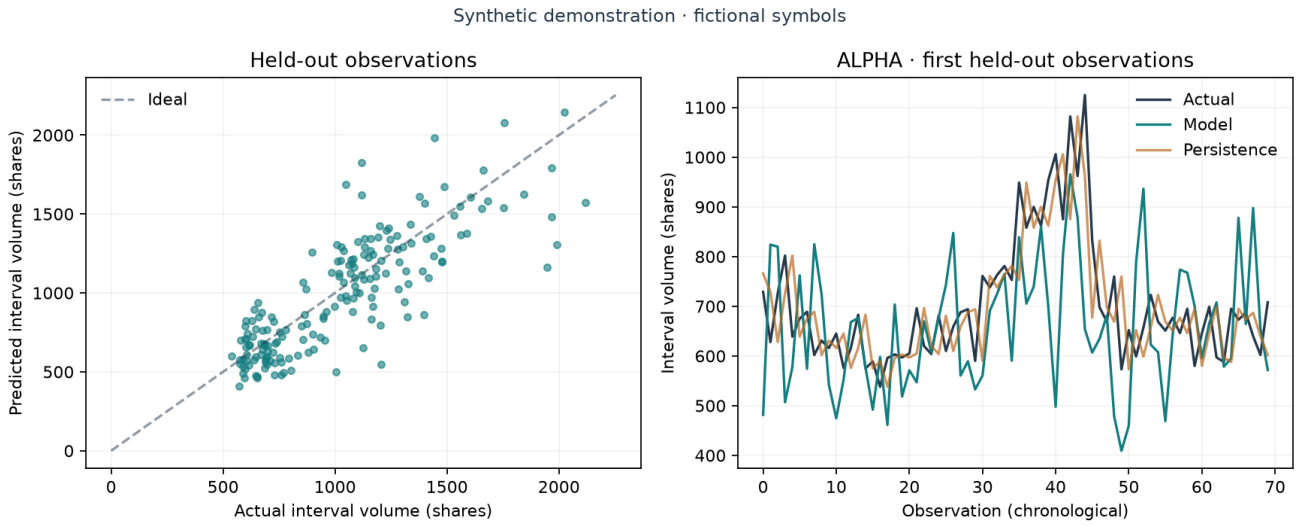


Figure 2. Held-out synthetic observations, model predictions, and persistence. The supplied historical charts and scores were not reused as current results.

Source: complete result snapshot. Run ID: e6d8efd4da9249249b41685cc62209ed.

05 / PROJECT REPORT

Reproducibility and code quality

The full test suite contains 30 passing tests, including a real TensorFlow training and save/reload check. Numerical tests cover counter allocation, daily resets, feed gaps, causal windows, date splitting, training-only scaling, metric edge cases, and invalid configuration. Ruff verifies lint and formatting.

Recorded environment

Python 3.12.14; TensorFlow 2.20.0; Keras 3.15.1; NumPy 2.3.5; pandas 3.0.1; scikit-learn 1.9.1; Matplotlib 3.11.2. The full suite passed on Windows CPU and Ubuntu GitHub Actions. The environment emits upstream Keras/NumPy deprecation warnings without failed assertions.

Setup in PowerShell

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-lock.txt
.\.venv\Scripts\python.exe -m pytest
.\.venv\Scripts\python.exe -m ruff check .
```

Run the included sample

```
.\.venv\Scripts\python.exe volume_lab.py train `
  --data data/sample --output artifacts/demo --epochs 5 `
  --data-label "Synthetic demonstration - fictional symbols"
```

For the notebook workflow, select the project environment, execute all cells in order, and run tools/export_report.py. The exporter checks that notebook outputs, saved results, and the implementation hash belong to the same run. The README gives the complete kernel and export commands.

Saved evidence

Each run records configuration, library versions, seed, date boundaries, sample counts, input hashes, training history, predictions, and labeled figures. Local model and preprocessing files stay outside version control. Load local pickle artifacts only from a trusted run.

The static report is already exported. Preview it with **python -m http.server 8000 --bind 127.0.0.1 --directory site**. Vercel serves the site folder; deployment does not train the model.

References: README, verification record, and automated checks.

06 / PROJECT REPORT

Limitations and next steps

Boundaries of the current project

Boundary	Practical implication
Synthetic observations	Fictional instruments and eight short dates cannot establish real-market forecasting performance.
Session assumptions	The policy groups local calendar dates and expects monotonic counters inside a session. Overnight markets and intraday feed resets need a revised contract.
Feed completeness	Cumulative differences alone cannot prove that every trade or tick was observed.
Bounded memory	Tick data and feature arrays remain in memory. Large histories require partitioning and memory measurements.
Research scope	No live market connection, inference service, trade execution, or CMEM residual estimator is included.

Recommended extension sequence

1. Obtain an authorized real-market dataset and verify its session and counter semantics.
2. Add longer chronological holdouts, rolling-date evaluation, and appropriate seasonal baselines.
3. Tune model choices using training and validation data, retaining an untouched final test period.
4. Measure performance by instrument, volume regime, and session segment; report failed experiments as well as improvements.

Source context

This maintained portfolio edition adapts a supplied academic notebook. Its convolutional encoder, context branch, instrument embedding, and dense head retain that architectural starting point. The edition consolidates the code, improves validation and temporal handling, and adds testing, reproducibility, documentation, and a synthetic demonstration. DeepLOB and CMEM provide research context; they are not claimed as inventions of this edition.

Explore the project

GitHub: BikashCoder31/intraday-volume-lab
Live report: intraday-volume-lab.vercel.app
Data contract | Publication guide | Source context

Prepared by Bikash Kushwaha. Technical details and metrics correspond to the recorded synthetic result snapshot dated 20 October 2026.